

План разработки Telegram-VPN

Проект сочетает Telegram-бот на Python (асинхронный фреймворк `aiogram` ¹) с прокси-сервером Marzban на базе Xray-core ². Marzban поддерживает прокси-протокол VLESS и технологию Reality ³ ⁴, что соответствует требованиям проекта. Концепция **Zero Logs** означает полное отключение логирования в Xray (в конфиге нужно установить `log.access` и `log.error` в "none") ⁵ ⁶. Вместо SQLite для хранения данных пользователей будем использовать PostgreSQL (аналогично MySQL, что рекомендовано при большом числе юзеров ⁷).

В файле `.env` нужно указать ключи Telegram и адрес БД: например, `TELEGRAM_API_TOKEN` и `TELEGRAM_ADMIN_ID` ⁸ для работы бота, а также `SQLALCHEMY_DATABASE_URL=postgresql://user:pass@host/dbname` ⁹ для подключения к Postgres. Остальные сервисы (платежи, Xray и пр.) также получают свои переменные окружения. Далее будем поэтапно готовить проект.

Шаги

1. **Запустите Claude Code в проекте.** В терминале перейдите в папку проекта и выполните команду `claude` ¹⁰.
2. *Claude Code:* создаст новый чат (или сессию) в режиме Plan Mode.
3. *Вы:* скопируйте в этот чат указанный контекст (текст промпта с ролями и задачами) и отправьте его. Это задаст ИИ роль ведущего разработчика.
4. **Создание структуры проекта.** Попросите Claude Code спланировать архитектуру проекта. Лучше начать с планирования (Plan Mode), затем переключиться в нормальный режим.
5. *Claude Code:* сгенерирует папки и файлы по технологии (например, создаст папки `bot/`, `webapp/`, `infra/` и основные скрипты).
6. *Вы:* проверьте предложенную структуру. Убедитесь, что есть: `bot/` (код бота на `aiogram`), `webapp/` (мини-приложение на React+Vite+Tailwind), `infra/` (Docker-файлы и конфиги). При необходимости скорректируйте (переименуйте, добавьте README и пр.).
7. **Настройка файла .env.** В каждом компоненте нужен свой `.env`. Начните с основного:
8. *Claude Code:* заполнит шаблон `.env` с переменными для сервиса. Он может указать, например, `TELEGRAM_API_TOKEN=`, `TELEGRAM_ADMIN_ID=`, `SQLALCHEMY_DATABASE_URL=` и др. (см. документацию Marzban ⁸ ⁹).
9. *Вы:* внесите реальные значения: получите токен бота у BotFather, ваш Telegram ID (например, через `@userinfobot`) и заполните их. Задайте параметры подключения к Postgres. По желанию добавьте переменные для проху, платёжных шлюзов (например, `ROBOKASSA_ID`, `ROBOKASSA_SECRET` и т.д.) по аналогии.

10. **Создание файла CLAUDE.md.** Этот файл хранит правила и соглашения для ИИ.
11. *Claude Code:* создаст пример CLAUDE.md по шаблону: опишите стиль кода (Python 3.x, PEP8, aiogram), правила работы (фиксация окружения, команды запуска, префиксы веток и т.д.).
12. *Вы:* скорректируйте его под проект. Укажите, что используем aiogram 3.x, SQLAlchemy, линтеры, форматирование кода. Добавьте заметки о предпочтительном workflow (напр., сначала план, затем код, запуск тестов).
13. **Разработка бэкенда (бота).** Реализуйте основы логики бота в папке `bot/`.
14. *Claude Code:* сгенерирует скелет бота на aiogram: инициализацию `Bot(token)`, диспетчер, пример обработчиков команд `/start`. Добавит подключение к базе через SQLAlchemy.
15. *Вы:* проверьте и доработайте. Настройте хендлер для автоматической выдачи 24-часового доступа (сохраните в БД), реализуйте логику реферальной системы, подключите платёжные методы (Telegram Stars, Robokassa). При необходимости подправьте код и протестируйте его с помощью unit-тестов или эмуляции.
16. **Разработка фронтенда (Telegram Mini App).** Настройте `webapp/` для мини-приложения на React.
17. *Claude Code:* создаст проект через Vite с React и TailwindCSS, подключит библиотеку `@tma.js/sdk`. Добавит базовые компоненты (авторизация через Telegram WebApp, интерфейс «Glassmorphism»).
18. *Вы:* реализуйте стили в духе glassmorphism и настройте Haptic Feedback. Спроектируйте страницы для отображения статуса подписки, кнопки оплаты и управления аккаунтом. Проверьте взаимодействие mini-app с ботом.
19. **Инфраструктура и Docker.** Настройте окружение `infra/`: образ Xray/Marzbان и сопутствующие скрипты.
20. *Claude Code:* создаст `docker-compose.yml` и/или Dockerfile для Marzbان и Xray. Добавит конфиги Xray с протоколом VLESS+Reality и параметрами Zero Logs (в `xray_config.json` добавит `"access": "none"`, `"error": "none"` ⁵ ⁶). Возможно, оформит Dockerfile для Marzbان по быстрому инсталлятору (или покажет команды установки).
21. *Вы:* запустите эти контейнеры на сервере (Debian 12). Убедитесь, что Marzbан поднялся (по умолчанию на порту 8000 ¹¹) и что Xray готов принимать подключения VLESS+Reality. Проверьте, что нулевое логирование работает (в папках логов нет файлов). Настройте переменные окружения для контейнеров, сделайте перезагрузку.
22. **Завершение и тестирование.** После сборки системы убедитесь, что всё работает как нужно:
23. *Вы:* проведите интеграционное тестирование: отправьте `/start` боту, проверьте выдачу тестового периода, создайте платную подписку через Robokassa, просмотрите статус в

мини-приложении. Настройте логи и мониторинг. При необходимости скорректируйте код (вы можете снова обратиться к Claude Code для исправлений).

24. *Claude Code*: по запросу может сгенерировать дополнительные сценарии, тесты или исправления в коде (например, проверить граничные случаи или оформить скрипты деплоя). По готовности попросите ИИ сделать коммит с описанием изменений.

После этих шагов у вас будет готова базовая структура проекта и начальный код. Далее можно добавлять оставшиеся фичи (реферальная система, доп. платежные шлюзы) аналогичным образом. Самые трудоёмкие шаги (архитектура, начальный код, Docker) рекомендовано поручить Claude Code, а вы в свою очередь вносите ключи, правите дизайн и тестируете конечную работу.

Пример структуры папок:

```
project_root/
├─ bot/
│  ├─ main.py
│  ├─ handlers/
│  └─ utils/
├─ webapp/
│  ├─ src/
│  ├─ public/
│  └─ tailwind.config.js
├─ infra/
│  ├─ docker-compose.yml
│  ├─ xray_config.json
│  └─ deploy.sh
└─ CLAUDE.md
```

Каждый шаг должен быть четко согласован: в начале задавайте Claude Code конкретную задачу (например, «создай файл .env с такими-то переменными»), а потом проверяйте результат. Так вы начнете работать над проектом, разделяя обязанности между собой (админом/ПО) и ИИ-ассистентом. Удачи в разработке!

-
- 1 aiogram 3.27.0 documentation

<https://docs.aiogram.dev/en/v3.27.0/>

- 2 3 4 Introduction to Marzban | Marzban

<https://gozargah.github.io/marzban/en/docs/introduction>

- 5 6 Log Configuration | Project X

<https://xtls.github.io/en/config/log.html>

- 7 Marzban Script | Marzban

<https://gozargah.github.io/marzban/en/docs/marzban-script>

- 8 9 Configuration | Marzban

<https://gozargah.github.io/marzban/en/docs/configuration>

- 10 Claude Code overview - Claude Code Docs

<https://code.claude.com/docs/en/overview>

11 Installation | Marzban

<https://gozargah.github.io/marzban/en/docs/installation>